

電腦網路 Computer Network Homework 2

Department of Computer Science and Engineering,
National Sun Yat-sen University

Asst. Prof. ShengMin(Sam) Chiu

Homework 2

- Implementation of Mini File Transfer Protocol (**MFTP**) and using Wireshark to analyze it.
- In this assignment, we will implement a custom **application-layer protocol** called **MFTP** using Python. Unlike Assignment 1, which involved simple string transmissions, the core challenge of this task is to handle the characteristics of **TCP Byte Streams** and ensure data integrity during the transfer of large files.
- MFTP Protocol Specification
 - Every message transmitted via MFTP must include a **7-byte fixed header**.
 - **(Important)** The header must be packed using **Network Byte Order (Big-Endian)**.
 - The structure is defined as follows:

Field Name	Length (Bytes)	Format Symbol	Description
Magic Number	2	H	Fixed as 0x4D46 ('M' and 'F' in ASCII).
Command	1	B	Command Code (see table below).
Payload Length	4	I	The total length of the following data segment (Payload).

Implementation Constraints & Requirements

- Strict Buffer Size Constraint
 - For all `recv()` calls, you MUST use a fixed buffer size where `BUFFER_SIZE = 4096`.
 - You are strictly prohibited from using a large buffer (e.g., `recv(payload_length)`) to receive an entire file in one go.
- Byte Stream Handling
 - You better to implement a `recv_all(count)` logic.
 - The program should first read exactly 7 bytes to parse the header, then enter a loop to call `recv(4096)` repeatedly until the total bytes received match the Payload Length.
- There is a
 - `./client_dir/` directory in client-side.
 - `./server_dir/` directory in server-side.

Implementation Constraints & Requirements

■ Client-side Functions

- **ls**: Send a List request. Receive and print the file list returned by the server.
- **get <filename>**: Send a Get request. If successful, download the file and save it to the ./client_dir/ directory. If it fails, display the error message returned by the server.
- **quit**: Send a Quit request, gracefully close the connection, and exit the program.

■ Server-side Functions

- Serve files only from the ./server_dir/ folder.
- Handle file transfers using binary mode (i.e. 'rb', 'wb').
- sent_all() is allowed to use in server-side.
- The server only needs to serve one client at a time for this assignment.

Command Codes

- 0x01: List (Client requests the list of available files)
- 0x02: Get (Client requests a file download; Payload contains the filename)
- 0x03: Quit (Client requests to terminate the connection)
- 0x04: Data (Server returns data; Payload contains the **file list string** or **binary file data**)

Reminders

- Header Packing and Parsing (using *struct* module)
 - Use Python's *struct* module to ensure correct byte ordering across different platforms.

```
import struct

# Packing the Header (7 Bytes)
# !: Network Byte Order, H: 2 bytes, B: 1 byte, I: 4 bytes
header = struct.pack('!HBI', 0x4D46, 0x02, len(payload))

# Unpacking the Header
magic, cmd, length = struct.unpack('!HBI', header_data)
```

位元組序 Endianness

- E.g., 0x12345678
- Big-Endian(Network Byte Order)
 - → 12, 34, 56, 78
 - Using in Internet protocols, such as TCP/IP header.
- Little-Endian
 - → 78, 56, 34, 12
 - Most Intel & AMD x86/x64 CPU using it.

```
import struct

# Packing the Header (7 Bytes)
# !: Network Byte Order, H: 2 bytes, B: 1 byte, I: 4 bytes
header = struct.pack('!HBI', 0x4D46, 0x02, len(payload))

# Unpacking the Header
magic, cmd, length = struct.unpack('!HBI', header_data)
```

Personalized Testing Task

- Setup: Download 10 images in eLearning platform (named test_0.jpg to test_9.jpg), and place them to ./server_dir/
- From ls to quit, randomly testing your program (including at least once ls and get), for your Get test case, you MUST include getting the **test_X.jpg** where **X is the last digit of your Student ID**
- Verify that the downloaded images open correctly and are not corrupted.

Wireshark Analysis & Report

- Use Wireshark to capture a complete MFTP session and provide a detailed analytical report. You must execute all three client actions at least once in a single session, including the download of the image corresponding to the last digit of your student ID (test_X.jpg).
- Annotated Time Sequence Diagram: Draw a complete Time Sequence Diagram covering the entire process (from the TCP 3-way handshake to connection close).
- Your diagram must include:
 - Labels & Descriptions: Identify each packet, correlating MFTP commands with their corresponding transport-layer behaviors.
 - Note: For all packets with an MFTP header, highlight the 7-byte header and explain the protocol interaction.

Submission

- Please **STRICTLY** follow the naming conventions and formatting requirements below for your submission
- Source Code:
 - Server-side: [ID]_hw2_server.py
 - Client-side: [ID]_hw2_client.py
- Report: Organize your report in **slide format** and **export it as a PDF file**.
 - File name: [ID]_hw2.pdf
- Final Package: Compress the three files above into a single ZIP archive.
 - File name: [ID]_hw2.zip
- Note: Replace [ID] with your actual student ID (e.g., B12345678).
- Deadline: 2 weeks